

LLMsVerifier

LLMsVerifier

Тексеру. Бақылау. Оптималдау.

Қысқаша мазмұны

LLMsVerifier – әртүрлі провайдерлер бойынша Зур Тіл Модельдерін (ЗТМ) тексеру, мониторингтеу және оптималдау үшін арналған кәсіпорын деңгейіндегі платформа. Әрбір модель қолданысқа жіберілмес бұрын міндетті “Менің кодымды көресің бе?” тексеру тестінен өтуі керек. Тек нақты жұмыс істейтіні дәлелденген модельдер ғана “пайдалануға жарамды” деп белгіленіп, экспортталады.

Қысқаша сипаттама

LLMsVerifier платформасы әртүрлі провайдерлер бойынша ЗТМ-ді тексеру, бенчмарктеу, мониторингтеу және оптималдауды жүзеге асырады. Әрбір модель қолданысқа жіберілмес бұрын міндетті кодтың көрінуін тексеру тестінен өтуі қажет. Осыдан кейін модель кешігу уақытын, ағынды беруді, функция шақыруды, бейнелерді өңдеуді және эмбеддингтерді тексереді, содан кейін тек тексерілген конфигурацияларды AI және CLI құралдарына экспорттайды.

Толық сипаттама

LLMsVerifier – әртүрлі провайдерлер бойынша LLM өнімділігін тексеру, мониторингтеу және оптималдау үшін кешенді платформа. Оның негізгі қағидасы – *міндетті тексеру*, және ол осы принциптен ешқандай ауытқуға жол бермейді: кез келген модель “пайдалануға жарамды” деп белгіленбес бұрын – немесе экспорттық конфигурацияға енгізілмес бұрын – ол “Менің кодымды көресің бе?” тестінен өтуі тиіс. Бұл тест провайдерге нақты HTTP сұраныстарын жібереді де, жауапты нақты түсіну үшін талдайды, тек сенімді көрініп тұрған

жауапты емес. Сіздің енгізіліміңізді көрсете алмайтын және түсінетін модель ешқашан “пайдалануға жарамды” белгісін ала алмайды. Осы шлюзден өткеннен кейін Тексеру Двигателі модельдің барлық мүмкіндіктерін тексереді – болуы, жауап беру жылдамдығы, кешігу уақыты, ағынды беру, функция шақыру, бейнелерді өңдеу, embeddings – ал Есептеу Двигателі нәтижелерді markdown және JSON есептерге айналдырады, оларды іске асыруға болады.

Жүйе модульдік және оқиғаларға негізделген, CLI, ТЖК (Терминалдық қолданушы интерфейсі), Веб және REST API интерфейстерін Тексеру Двигателі, Есептеу Двигателі және Конфигурация Менеджері негізінде ұсынады. Ол тек тексерумен шектелмейді. Жетілдірілген қабаттар LLM-ға негізделген тапсырмаларды бөлу үшін Супервайзер/Жұмысшы үлгісін, өте ұзақ сессиялар құлдырамайтындай жылжымалы терезелер мен LLM-қорытындылау контекстін басқаруды, бұлттық нүктелерді сақтауды және ақауларға төтеп беру жүйесін қосады. Жүйеге қосымша кедергілерді ұзу және кешігуге негізделген маршруттау да енгізілген. Қоршаған инфрақұрылым өндіріс деңгейінде жасалған: pub/sub оқиғалар магистралі, cron жоспарлауы, баға/шектеулерді анықтау, RAG үшін vector дерекқоры және экспорт жүйесі. Әрбір жасалған провайдер/модельге (llmsvd) қосымшасы қосылады, сондықтан тексерілген шығарылым бір қарағанда анықталады да, тексерілмегендермен шатастырылмайды. Тек тексерілген модельдер ғана AI CLI құралдарына экспорттық конфигурацияларға жазылады, мысалы, OpenCode, Crush және Claude Code. Платформа өндірісте командаларға қажетті операциялық құралдармен бірге жеткізіледі: Docker/ Kubernetes/Helm орналастыру, Prometheus/Grafana мониторингі, LDAP/SSO және SQLCipher-шифрланған сақтау.

Неліктен жасадық

Себебі конфигурацияны тек тексеру сенімсіз – API кілтінің мерзімі біткен болуы, модель ескірген болуы, ал конфигурациялық файл нақты кешігу, нақты қателер немесе модельдің шынымен сіздің енгізіліміңізді көре алатынын және түсіне алатынын айтпайды. LLMsVerifier “ол конфигурацияда болса, онда жұмыс істейді” дегенді дәлелмен ауыстырады: тек нақты дұрыс жауап беретін модельдер ғана “пайдалануға жарамды” деп белгіленіп, экспортталады.

Мазмұны

Неге бұл ойынды өзгертетін шешім

Ол LLM паркін *сенімді* етеді — бұл кеңістіктегі конфигурациялардың көбі көмескі түрде жалған айтатын жерде сирек кездесетін сөз. Құрылған модель жұмыс істейді деп үміттенудің орнына, командалар әрбір қолданыстағы модель нақты тексерістен өткеніне кепілдік алады, мониторинг, апаттық ауыстыру және тексерілгендерді ғана экспорттау дәлелденгеннен өндіріске дейінгі циклді жабады. Helix экосистемасында ол LLM моделі, провайдері және тексеру метадеректері үшін жалғыз шындық көзіне айналады: басқа қызметтер (оның ішінде HelixTranslate) оған сілтейді, сондықтан бүкіл платформа «қазір қай модельдер шынымен жұмыс істейді?» деген сұраққа бір ғана шындықты жауап алады, әр команда өз үмітінен болжау жасаудың орнына.

Инновациялық жақтары

- **Міндетті «Менің кодымды көресің бе?» тексеруі** — модель қолданысқа енгізілмес бұрын өтуі керек нақты, HTTP-негізіндегі түсіну қақпасы; өнімнің ерекше айырмашылығы және тексерілмеген ештеңе өтпейтін себебі.
- **Тек тексерілген конфигурацияларды экспорттау** — AI CLI құралдары үшін жасалған конфигурацияларда *тек* тексерістен өткен модельдер ғана болады, сондықтан жіберілетін конфигурацияда сынған модельді тыныштықпен қайта енгізу мүмкін емес.
- **(llmsvd) бренд-суффикс жүйесі** — әрбір жасалған провайдер/модельде іздерді бақылауға болатын суффикс бар, бұл тексерілген шығу тегін барлық шығарылымдарда көрінетін етеді.
- **CLI агенттері мен провайдерлері бойынша мүмкіндіктерді анықтау** — ол стриминг түрлерін (SSE, WebSocket, JSONL, EventStream), сығу және кәштену әрекеттерін саусақпен басып анықтайды, оларды болжаудың орнына.
- **Өтімді апаттық ауыстыру** — схема бұзылғыштары, уақыт-бірінші-белгіге дейінгі мерзім шегінен асып кеткенде қайта бағыттау жасайтын кешігуге негізделген маршруттау, денсаулықты тексеру зондтары және салмақты трафик бөлінісі жеке провайдерлердің дөңгелеген кезінде паркті жауапты етеді.
- **Ұзақ мерзімді автономия** — Супервайзер/Жұмысшы декомпозициялау үлгісі, үзілістерден аман қалу үшін бұлттық сақтауға кезеңдік чекпоинттер және контексті басқарудың қабаттасқан жүйесі (жылжымалы терезе + LLM қорытындылау + RAG) модельдің тақырыпты ұмытпауына және токендерге батып кетпеуіне көмектеседі.

- Негізделген контекстті жақсарту үшін RAG / vector-DB интеграциясы.

Ең үлкен техникалық қиындықтар және оларды қалай шештік

- **Модель шынымен жұмыс істейтінін дәлелдеу, тек конфигурацияланғаны емес.** Бұл негізгі мақсат және ең қиын бөлігі. Нақты API шақыруларын жасап, жауаптарды түсіну үшін талдайтын міндетті кодты көру сынағынан өткізу арқылы шешілді, мүмкіндіктерді кеңінен тексеру жиынтығымен қолдау көрсетілді — содан кейін тексерістен өтпеген ештеңені экспорттауға рұқсат берілмеді, сондықтан өндірістің қақпасы конфигурация емес, дәлел болды.
- **Көптеген сенімсіз үшінші тарап провайдерлері бойынша сенімділік.** Провайдер тұрақсыздығын қалыпты жағдай деп қарайтын апаттық ауыстыру оркестраторы арқылы шешілді: схема бұзылғыштары N істен шығуды M секунд ішінде белгілейді, кешігуге негізделген маршруттау баяу нүктелерден аулақтайды, кезеңдік денсаулықты тексеру қалпына келуді тексереді, ал салмақты маршруттау арзан және премиум модельдер арасындағы тепе-теңдікті сақтайды.
- **Өте ұзақ, автономды сессияларды қолдау.** Үлкен жұмысты іске асырылатын бөліктерге бөлетін Супервайзер/Жұмысшы декомпозициялау үлгісі арқылы шешілді, үзілістерден аман қалу үшін бұлттық сақтауға кезеңдік чекпоинттер және контекстті басқарудың қабаттасқан жүйесі (жылжымалы терезе + LLM қорытындылау + RAG) модельдің тақырыпты ұмытпауына және токендерге батып кетпеуіне көмектеседі.
- **Провайдерлердің көптігі.** Бір ортақ интерфейс артында көптеген әр провайдерлік Go адаптерлерін жасыру арқылы шешілді, ал нақты нүктелер орталықтан тізімделеді — сондықтан провайдерді қосу шектеулі өзгеріс болып табылады, код базасы бойынша толқын тудырмайды.

Мазмұн

Технологиялық стек

- **Go** — негізгі платформа тілі ретінде таңдалды, себебі ол параллельділікті жақсы қолдайды; көп ағынды Верификаторлық Қозғалтқышты басқарады, ол бір мезгілде көптеген модельдерді тексеруге мүмкіндік береді, сонымен қатар қосымша қызметтерді де.

- **Gin** — REST API сервері ретінде таңдалды, JWT аутентификациясын, жылдамдықты шектеуді және WebSocket/SSE шлюздерін қамтиды.
- **SQLite + SQLCipher** — дерекқор деңгейіндегі шифрлаумен кірістірілген сақтау үшін таңдалды, себебі верификациялық деректер (күдіктер, нәтижелер) сезімтал болып табылады және әдепкі бойынша тыныштық күйінде шифрлануы керек.
- **Redis** — кеш қабаты ретінде таңдалды, верификация мен метадеректерді жылдам іздеуді қамтамасыз ету үшін.
- **RabbitMQ + Kafka** — платформадағы өндірушілер мен тұтынушыларды бөліп тұратын хабарламалар мен ағындарды қамтитын оқиғаларға негізделген архитектураны іске асыру үшін таңдалды.
- **gRPC + Protocol Buffers** — компоненттер арасындағы қызметтер арасындағы байланыс пен оқиғаларды тасымалдау үшін қатаң түрлендірілген байланыс үшін таңдалды.
- **QUIC / HTTP-3 (quic-go)** — қазіргі заманғы тасымалдауды қолдау үшін таңдалды (репозиторий құжаттарында HTTP/3 провайдерлерінің қолжетімділігі шектеулі деп көрсетілген — бұл мүмкіндік ұсынылғаны, жалпыға бірдей мәлімдеме емес).
- **JWT + LDAP/NTLM** — корпоративтік аутентификация үшін таңдалды, бұл платформаның бар корпоративтік идентификация жүйесіне (SSO/SAML/OIDC құжатта көрсетілген) сәйкес келуіне мүмкіндік береді.
- **Viper (конфигурация), Logrus (күнделік жазу), Brotli/compress (сығылу)** — жұмыс процестері: икемді конфигурациялау, құрылымдық күнделіктер және деректерді сығу.
- **Angular** — веб-қосымшаның бір беттік қолданбасы ретінде таңдалды, бұл верификация мен мониторингке кірудің визуалды қақпасы.
- **Python + JavaScript SDK** — клиенттік командаларға бірінші дәрежелі қол жеткізу мүмкіндігін беру үшін таңдалды, OpenAPI/Swagger арқылы құжатталған.
- **Docker, Kubernetes, Helm** — өндірістік орналастыру үшін таңдалды, денсаулықты бақылау мен автоматты масштабтауды қамтиды, бұл верификациялық паркті қазіргі заманғы қызмет сияқты масштабтауға мүмкіндік береді.
- **Prometheus + Grafana** — көрсеткіштер мен бақылау тақтасын қамтамасыз ету үшін таңдалды, бұл платформаның өз денсаулығын оның бақылайтын модельдері сияқты бақылауға мүмкіндік береді.
- **Testify (Go) + node -test/jsdom (веб)** — Go ядросы мен веб-алдыңғы қабатындағы қабатты тестілеу үшін таңдалды.

Жағдай және ашықтық ескертпелері

- **Жағдайы: бета.** Go бастапқы коды нақты HTTP верификациясын іске асырады (верификацияны тек конфигурация арқылы іске асыруды сипаттайтын ескі құжат — бұл көзқарастық және ескірген, код — ресми дереккөз).
- **Лицензиясы: анықталмаған.** README файлы MIT деп көрсетеді, ал Dockerfile белгісі Apache-2.0 деп көрсетеді — жарияламас бұрын шешім қабылдау қажет.
- **Провайдерлер саны:** README файлында “12 адаптер” деп көрсетілген, бірақ провайдерлер каталогында шамамен 26 бар — оны “12+ / жұмыс үстінде” деп қарастыру керек. Көптеген “ТҰЫНДЫ/АЯҚТАЛҒАН” мәртебелі файлдар бар; код, құжаттар және go.mod ресми дереккөз болып табылады.
- Репозиторий vasic-digital ұйымында орналасқан, бірақ іс жүзінде ол Helix LLM-инфрақұрылымдық кластерінің сенім қабаты болып табылады.

Басымдық деңгейі: Helix-негізгі (LLM-инфрақұрылымдық кластер; LLM/провайдер/верификация метадеректері үшін бірден-бір шындық көзі). HelixTrack кейін орналасады.